Technische Hochschule Ingolstadt

Bachelor Thesis

in the study program Computer Science and AI
Faculty of Computer Science

Enhancing Software Design Quality through AI-based Structural and Semantic Validation of Software Architecture Documentation

First- and Last name : **Hana Ben Slama**

Registered on : 05.11.2025

Submitted on : 28.02.2026

First Examiner : Prof. Dr.Bernd Hafenrichter

Second Examiner : Prof. Dr. Ludwig Maximilian Lausser

Company advisor : Mr. Martin Schwarz
Company name : e.solutions GmbH

Affidavit

I declare that I have authored this thesis independently, that I have not used other than the declared sources/resources, that I have not presented it elsewhere for examination purposes, and that I have explicitly indicated all material which has been quoted either literally or by consent from the sources used. I have marked verbatim and indirect quotations as such.

Ingolstadt, _____

Firstname Lastname

Acknowledgments

This is the sentimental part where you get to thank all the persons who were a part of your thesis journey in one or the other way!

Hana Ben Slama
Ingolstadt, Germany
February 28 2026

Abstract

Here goes a brief description of your entire thesis.

Confidentiality clause

Optional.

Ingolstadt, _____
(Date)

(Signature)
Firstname Lastname

List of figures

List of tables

Table of contents

++

1 Introduction

Software architecture documentation plays a central role in communicating design intent across stakeholders and throughout system evolution. International guidance on architecture description emphasizes that architectural documentation should address stakeholder concerns through multiple views and viewpoints, rather than relying on a single representation **?**. In industrial practice, architecture documentation often serves as the primary artifact for communicating architectural decisions, constraints, interfaces, and system decomposition across teams and organizational boundaries.

However, architecture documentation quality is difficult to maintain over time. Empirical studies report that architectural inconsistencies evolve during system evolution and that architecture descriptions are not always trusted or used to the extent intended, which undermines their value as engineering artifacts **?**. This challenge is compounded by documentation-related debt: incomplete, outdated, or low-quality documentation accumulates under time pressure and organizational constraints, reducing maintainability and increasing coordination cost **?**.

This thesis addresses these challenges by designing and implementing an AI-based validation pipeline for architecture documentation, focusing on structural and semantic validation against an arc42-derived reference template used in an industrial environment.

1.1 Motivation

Many organizations standardize architecture documentation via templates to improve consistency and reduce ambiguity. arc42 is a widely used template that structures architecture documentation into a set of sections representing complementary views (e.g., building block view, runtime view, deployment view) and provides guidance for what each section should contain **?**. Such structure supports engineering review and enables reviewers to locate architectural information efficiently.

Despite this, real-world documentation frequently deviates from templates due to renamed headings, partial adoption of sections, copied template guidance, or incomplete placeholders (e.g., "TODO" markers and unfilled slots). These deviations lead to two practical problems: (i) manual reviews do not scale across many documents and teams, and (ii) purely structural checklist validation is insufficient because it cannot assess whether the content is meaningful or consistent across views. At the same time, fully unconstrained semantic analysis is difficult to operationalize because outputs must remain verifiable and auditable for industrial acceptance.

The motivation of this work is therefore to provide an approach that detects documentation quality issues systematically while producing actionable, evidence-backed findings that can be integrated into existing workflows.

1.2 Problem Statement

The industrial context of this thesis maintains architecture documentation in AsciiDoc and uses an arc42-derived company template as a normative baseline. Documentation is processed at scale (e.g., per software unit) through automated tooling that converts documents into PDFs and produces aggregated statistics. However, comprehensive validation is limited in scope and significant portions of documentation quality assurance still rely on manual review or implicit trust.

The core problem addressed in this thesis is:

How can architecture documentation be validated automatically for both structural compliance and semantic adequacy, while remaining robust to real-world variation and producing auditable results suitable for industrial review?

This problem includes three key challenges:

1. **Structural variation in practice:** Headings and section names may differ from the template (e.g., renames, merged/split sections), which can cause strict structural checks to over-report missing content.
2. **False completeness:** Template remnants and placeholders can produce a structurally complete document that is not genuinely filled with architectural information.
3. **Cross-view coherence:** Architecture documentation is multi-view by design; inconsistencies across views (e.g., runtime scenarios referencing undefined building blocks) reduce trust and usefulness **??**.

1.3 Objectives and Research Questions

To address the problem, this thesis develops a staged validation approach that operationalizes documentation quality into checkable dimensions and produces structured outputs.

Objective O1 — Structural validation: Validate completeness and structural conformity against the company's arc42-derived reference template, including missing and extra sections and heading hierarchy.

Objective O2 — Integrity validation: Detect incomplete or misleading content (stubs, placeholders, copied template instructions) that undermines the usefulness of documentation.

Objective O3 — Semantic validation: Assess whether section content fulfills the intent expressed by the template guidance (e.g., whether the runtime view describes scenarios and interactions).

Objective O4 — Cross-view consistency validation: Detect inconsistencies across views by extracting architectural entities and checking their coherent usage across building block, runtime, deployment, decisions, goals, and risks.

These objectives are captured in the following research questions:

- **RQ1 (Structural):** How effectively can a deterministic baseline checker validate arc42-derived structure and detect missing/extra headings in real industrial documents?
- **RQ2 (Robustness):** How can similarity-based alignment reduce false "missing section" findings caused by heading renames and variations?
- **RQ3 (Semantic & Integrity):** To what extent can an LLM-based audit, constrained to evidence-backed outputs, detect stubs and assess semantic adequacy relative to arc42 guidance?
- **RQ4 (Consistency):** How can cross-view consistency checks be operationalized on text-centric arc42 documentation without assuming formal architecture models?

1.4 Approach Overview

This thesis proposes a staged pipeline that combines deterministic validation, similarity-based alignment, and constrained semantic auditing:

1. **Parsing and normalization:** Convert AsciiDoc documents into a normalized JSON representation of sections, headings, and content blocks.
2. **Baseline structural validation:** Deterministically compare document structure against the reference template to identify missing and extra elements.
3. **Similarity-based resolver:** Apply TF-IDF and cosine similarity to propose correspondences between template headings and document headings when naming variation exists.
4. **Preprocessing layer:** Build an audit-ready representation by aligning sections/headings and attaching template guidance for each section to the corresponding content.
5. **Pass 1 audit (integrity + semantic adequacy):** Evaluate whether headings contain real content (integrity) and whether content satisfies guidance intent (semantic adequacy), producing evidence-backed findings.
6. **Pass 2 audit (cross-view consistency):** Extract architectural entities and detect inconsistencies across views (e.g., runtime scenarios referencing undefined building blocks or missing deployment mappings).

The design is grounded in the multi-view nature of architecture description and the intent of arc42 sections: the building block view provides static decomposition, the runtime view describes behavioral scenarios, and the deployment view documents infrastructure and mapping **?**.

1.5 Contributions

This thesis makes the following contributions:

1. A documentation quality model tailored to arc42-based validation that operationalizes documentation quality into structural completeness, integrity, semantic adequacy, and cross-view consistency.
2. A deterministic baseline checker that validates arc42-derived structure and provides explainable outputs for missing/extra sections and heading hierarchy deviations.
3. A similarity-based alignment layer that rescues template-to-document matches under heading/name variation without treating similarity as a proxy for quality.
4. A guidance-aware preprocessing representation that couples each section with its expected intent to support auditable semantic validation.
5. A constrained semantic auditing design (Pass 1) and a consistency auditing design (Pass 2) producing evidence-backed, structured JSON outputs for downstream aggregation and review.

Together, these contributions target an industrially relevant gap: comprehensive, auditable validation of text-centric arc42 documentation that goes beyond section presence checks and supports scalable review.

1.6 Thesis Structure

The remainder of this thesis is organized as follows:

- **Section 2 (Background):** Introduces software architecture documentation fundamentals, arc42 as the documentation framework, and documentation quality dimensions used throughout the thesis.
- **Section 3 (Literature Analysis):** Reviews existing approaches for structural validation, similarity/IR-based alignment, ML/anomaly detection, and LLM-based semantic analysis, and derives the research gap and positioning of this work.
- **Section 4 (Industrial Context & Requirements):** Describes the industrial workflow and constraints and derives functional and non-functional requirements for the toolchain.
- **Section 5 (Methodology):** Presents the staged validation methodology, including pipeline design, quality model operationalization, and evaluation plan.
- **Section 6 (Implementation):** Details the implementation of parsing, baseline checks, similarity resolver, preprocessing, and the two-pass audit architecture, including reproducibility measures.
- **Section 7 (Evaluation):** Evaluates the pipeline on real industrial documents, compares the contribution of stages, and reports quantitative and qualitative findings.
- **Section 8 (Discussion):** Discusses strengths, limitations, industrial applicability, and threats to validity.
- **Section 9 (Conclusion & Future Work):** Summarizes contributions, answers the research questions, and outlines future extensions.

2 Background and Theory

This section introduces the theoretical foundations required for the remainder of the thesis. First, it summarizes core concepts of software architecture documentation, including stakeholder concerns, viewpoints, and the rationale for multi-view architecture descriptions. Second, it introduces arc42 as the documentation template family underlying the company-specific reference template used in this work. Finally, it defines the documentation quality dimensions operationalized by the proposed validation pipeline—structural completeness, integrity, semantic adequacy, and cross-view consistency—which are later used to derive requirements, guide implementation, and interpret evaluation results.

2.1 Software Architecture Documentation Fundamentals

Software architecture documentation captures architectural structures, decisions, and constraints in a form that supports communication, review, and evolution over time. A key framing is provided by ISO/IEC/IEEE 42010, which characterizes an *architecture description* as a structured set of information that identifies relevant stakeholders and their concerns and organizes the description into one or more *views* governed by corresponding *viewpoints* **??**. This perspective is particularly relevant in industrial contexts, where architecture documentation often serves as the primary boundary object across teams, roles, and organizational interfaces.

2.1.1 Stakeholders, concerns, viewpoints, and views

In ISO/IEC/IEEE 42010 terminology, a *stakeholder* is an individual, group, or organization with an interest in the system, and a *concern* represents an interest that is important from an architectural perspective (e.g., modifiability, deployment constraints, operational behavior, integration interfaces) **??**. A *viewpoint* specifies conventions for constructing and using a view, while a *view* presents architecture information addressing particular concerns under that viewpoint **?**. This decomposition is important because it makes explicit that architecture documentation is not a single artifact (such as one diagram), but rather a curated set of representations aimed at different questions and decision needs.

2.1.2 Rationale for multi-view architecture documentation

A central principle of architecture documentation is that no single representation can address all stakeholder concerns adequately. Multi-view documentation structures architectural knowledge into complementary slices and thereby improves navigability and reviewability. A widely cited example is Kruchten's "4+1" view model, which illustrates how multiple views (e.g., logical, development, process, physical) and scenario-based validation together provide a more complete and stakeholder-relevant architecture description than a single view **?**. More generally, multi-view approaches align with standards-based guidance: views exist to ensure that the documentation systematically addresses architecturally significant concerns for relevant stakeholders **?**.

2.1.3 Why architecture documentation quality degrades in practice

Even when templates and standards are used, documentation quality often degrades during system evolution. Empirical evidence from industrial surveys indicates that architecture documentation is frequently perceived as outdated, updated with delays, and inconsistent in form and detail, which reduces trust and limits practical use **?**. Such degradation can be understood as a form of documentation-related technical debt: documentation technical debt has been described as problems in documentation such as missing, inadequate, or incomplete artifacts that impose downstream costs on comprehension, coordination, and maintenance **?**. In industrial settings, this is reinforced by pragmatic constraints (time pressure, unclear

ownership, high variability across teams), which can lead to partially filled sections, residual template instructions, and unmaintained cross-references between views.

2.1.4 From documentation to validation needs

Because architecture documentation is both a communication artifact and an engineering control instrument, quality assessment must go beyond the presence of headings. Template-based documentation provides a strong baseline for structural completeness, but multi-view documentation introduces additional quality obligations: each view should fulfill its intended purpose, and views should be coherent with one another. The view/viewpoint framing in ISO/IEC/IEEE 42010 makes this explicit: views are included to address stakeholder concerns, which implies that quality must consider not only *structure* (whether required elements exist) but also *meaning* (whether the content actually addresses its intended concerns) and *consistency* (whether related views contradict each other) **?**. These needs directly motivate the staged validation approach in this thesis, which combines deterministic structural checks with guidance-aware semantic auditing and cross-view consistency validation.

2.2 arc42 as an Engineering Standard for Architecture Documentation

This thesis builds on arc42 as the conceptual and structural basis for the reference architecture documentation template used in the industrial case study. arc42 is a pragmatic, experience-based template intended to support the documentation and communication of software and system architectures across technologies and tooling environments **?**. Compared to informal and unstructured documentation, arc42 can be understood as an engineering-oriented contract: it defines which architectural concerns should be documented and where they belong, thereby enabling consistent communication and review across stakeholders.

2.2.1 Purpose and characteristics of arc42

arc42 is designed to make architecture documentation efficient and effective by providing (i) a standardized structure and (ii) embedded guidance on what to document in each part of the architecture description **??**. The template is intentionally practical: it aims to support understandability and adequacy of documentation without imposing unnecessary overhead. For this thesis, two characteristics are particularly relevant:

- **Template-driven structure:** arc42 defines a canonical hierarchy of sections and headings to ensure that the most important architectural concerns are addressed systematically.
- **Guidance as normative intent:** arc42 includes explanatory prompts and recommendations that describe the intended purpose of each section. These guidance texts can be interpreted as explicit expectations about the type of content a section should contain.

This combination makes arc42 suitable not only as an authoring aid, but also as a basis for systematic validation: structural checks can validate presence and hierarchy, while semantic checks can compare section content against its intended purpose.

2.2.2 Core structure and the arc42 section model

arc42 organizes architecture documentation into a sequence of sections that together cover the key concerns required to understand and assess a system architecture **?**. The canonical structure includes (among others) the following major sections: Introduction and Goals, Constraints, Scope and Context, Solution Strategy, Building Block View, Runtime View, Deployment View, Cross-cutting Concepts, Architecture Decisions, Risks and Technical Debt, and Glossary.

This structure aligns with common architectural reasoning processes:

- **Goals and constraints** define the drivers and boundaries of the system (why the architecture exists and which constraints must be respected).
- **Scope and context** positions the system relative to external actors and neighboring systems, including interfaces.
- **Solution strategy** captures overarching architectural ideas and high-level decisions.
- **Building Block View** describes static decomposition into building blocks and their dependencies.
- **Runtime View** complements static decomposition by documenting behavior and interactions through scenarios.
- **Deployment View** describes infrastructure and execution environments and—where applicable—maps software building blocks to infrastructure elements.

By combining complementary views, arc42 supports a holistic architecture description that is both navigable for stakeholders and reviewable for coherence.

2.2.3 Guidance as embedded quality expectations

A key feature of arc42 is that it not only defines headings but also provides guidance describing what should be included in each section **?**. For example, the Runtime View guidance describes scenario-based documentation of interactions between building blocks, such as important use cases, critical interface interactions, operational flows, and error scenarios. The Building Block View is framed as describing static decomposition and dependencies. The Deployment View guidance clarifies when deployment documentation is necessary and emphasizes documenting relevant environments and mappings.

In addition to high-level guidance, arc42 provides examples and practical tips. From the perspective of validation, these embedded instructions can be interpreted as quality expectations: they define what adequate section content typically looks like. This becomes foundational for semantic validation in later stages of the pipeline, where the document is checked not only for the presence of headings but also for whether the content fulfills the architectural purpose implied by arc42.

2.2.4 Tooling, formats, and single-source template maintenance

arc42 is available in multiple formats (e.g., AsciiDoc, Markdown, Word), which supports adoption in different toolchains **?**. The availability of a structured, machine-processable source format is relevant for this work: a stable template representation can be parsed, normalized, and used as a canonical reference for both structural validation and guidance extraction.

2.2.5 Implications for this thesis: arc42 as a validation backbone

arc42 is a suitable backbone for AI-based structural and semantic validation because it provides:

- **A canonical structure** (what should exist and where), enabling deterministic structural checks.
- **Explicit intent per section** through guidance, enabling semantic adequacy checks (content should fulfill the section's architectural purpose).
- **A multi-view organization** (static building blocks, dynamic runtime scenarios, and infrastructure/deployment), enabling cross-view consistency checks grounded in well-defined sections.
- **A structured, machine-processable template form**, enabling reproducible parsing and automated processing.

Template variant used in this thesis. While arc42 provides the conceptual foundation and canonical section model, the validation performed in this thesis is based on the *company-specific* arc42-derived template used in the industrial environment of the case study. This template follows the arc42 structure but may include adaptations such as organization-specific headings, additional constraints, or specialized guidance. Therefore, throughout this thesis, the term *reference template* refers to the company template as the normative baseline for validation, while arc42 is cited as the underlying documentation framework that motivates the structure, intent, and multi-view organization of the architecture documentation.

2.3 Quality Dimensions of Architecture Documentation

Architecture documentation is an engineering artifact that communicates architectural intent across stakeholders and across time. Even when a system is well designed, poor documentation can undermine implementation and evolution by causing misunderstandings, inconsistent decisions, and architectural drift. In industrial practice, architecture descriptions frequently become inconsistent or outdated, reducing their usefulness and leading practitioners to rely on informal knowledge instead of documented architecture. Wohlrab et al. report that architectural inconsistencies evolve during system evolution and that architecture descriptions are not always trusted or used to the extent intended **?**. In parallel, empirical work on documentation technical debt characterizes documentation shortcomings (e.g., non-existent, inadequate, or incomplete documentation) as a recurring form of debt that imposes downstream costs on comprehension and maintenance **?**.

This thesis aims to enhance software design quality by improving the reliability of the architectural knowledge captured in documentation. The claim is not that validation directly improves source code quality, but that it increases the dependability of the architecture description that guides design decisions, implementation, and review. To make this claim precise and evaluable, documentation quality is defined through operational dimensions that reflect both template compliance and architectural meaning.

2.3.1 Multi-view nature of architecture documentation

Architectural knowledge is inherently multi-perspective. ISO/IEC/IEEE 42010 formalizes architecture description in terms of stakeholders, concerns, viewpoints, and views, emphasizing that architecture should be described through multiple complementary representations rather than a single narrative **?**. arc42 follows the same principle by structuring documentation into sections that address different concerns (e.g., goals, constraints, context, building blocks, runtime scenarios, deployment, and rationale). Because architectural knowledge is distributed across multiple views, documentation quality must capture more than the existence of chapters. Quality must reflect whether the document is complete, whether each section fulfills its intended purpose, and whether the overall description remains coherent across views.

2.3.2 Structural completeness

Structural completeness describes whether an architecture document fulfills the structural obligations of its documentation framework. In this thesis, the arc42-derived reference template serves as the normative baseline: a document is structurally complete if all required sections (and expected heading hierarchies) are present and organized consistently with the template. Structural completeness is a necessary condition for quality because missing sections imply missing architectural concerns (e.g., an absent deployment view obscures operational assumptions). However, structural completeness alone cannot guarantee that the documented content is meaningful, which motivates additional quality dimensions.

2.3.3 Integrity of documented content

Integrity captures whether a section contains authentic, authored architectural content rather than incomplete or misleading artifacts (e.g., empty headings, placeholders, residual template instructions). Integrity matters because incomplete or template-like content can create a false impression of completeness: the structure exists, but the architectural knowledge is absent. The documentation debt perspective reinforces that such shortcomings are common and often persist unless systematically detected and addressed **?**. Therefore, integrity is treated as a distinct dimension: a document can be structurally complete while still exhibiting low integrity if its sections are not meaningfully authored.

2.3.4 Semantic adequacy to section intent

Semantic adequacy describes whether the content in a section fulfills the architectural purpose expected by the documentation framework. In arc42-derived templates, each section is associated with guidance that describes the intended content (e.g., runtime scenarios should describe interactions; building blocks should describe decomposition and dependencies; deployment should describe environments and mappings). A section may contain text yet still be semantically inadequate if it remains generic, irrelevant to the intended concern, or does not provide decision-relevant architectural information. Accordingly, this thesis treats semantic adequacy as a validation objective distinct from structural checks: it assesses whether documented content is fit for the architectural intent of the section, not simply whether text exists.

2.3.5 Cross-view consistency and intra-document traceability

Cross-view consistency captures whether the architecture description remains coherent across sections. Given the multi-view nature of architecture documentation, views should not contradict one another and should reference compatible architectural entities. For example, elements introduced in the building block view should provide the basis for runtime scenarios; deployment descriptions should account for major building blocks; design decisions should relate to documented goals; and risks should reflect documented trade-offs. As documented in empirical work, inconsistencies reduce trust and contribute to documentation underuse **?**.

A related research line demonstrates that mismatches across artifacts can be detected by recovering links between elements in different representations. For example, ArDoCo detects inconsistencies between natural-language software architecture documentation and formal architecture models by recovering trace links and flagging elements that appear in one artifact but not the other **?**. While this thesis does not assume the availability of formal architecture models, the underlying principle is directly relevant: documentation quality depends on identifiable relationships between architectural entities and their consistent usage across views.

Accordingly, this thesis uses the term *intra-document architectural traceability* to refer to recoverable relationships *within* an arc42 document (e.g., entities referenced across building block, runtime, and deployment views; rationale linked to goals and risks). Cross-view consistency is validated by reconstructing and checking these relationships.

2.3.6 From quality dimensions to measurable "enhancement"

To justify the thesis claim of "enhancing" design quality through validation, enhancement must be observable. Within the scope of this work, enhancement is defined as an increased ability to *detect and localize* documentation defects along the four quality dimensions above: structural completeness, integrity, semantic adequacy, and cross-view consistency. This definition supports evaluation in later sections by enabling stage-wise comparison (e.g., which defects are found by structural analysis alone versus which require semantic validation or cross-view consistency checks). The result is a documentation audit that

makes quality issues explicit and actionable, enabling architects and teams to improve the reliability of the architecture description over time.

2.4 Validation Terminology and Core Concepts

This section defines the key validation concepts used throughout the thesis. The goal is to establish precise terminology so that later sections (methodology, implementation, evaluation) can refer to a shared set of definitions without repeating explanations.

2.4.1 Structural validation

Structural validation assesses whether a document conforms to an expected structure. In the context of arc42-derived architecture documentation, this primarily includes:

- presence of required sections and mandatory headings,
- correctness of heading hierarchy (e.g., section–subsection nesting),
- detection of additional sections/headings that are not mapped to the reference template.

Structural validation is *normative*: it is evaluated against a reference template (Section 2.2) and does not require judging whether the content is meaningful. Its primary strengths are determinism and explainability (missing/extra elements are directly verifiable). Its main limitation is that structural conformance does not guarantee usefulness or completeness of the documented architectural knowledge.

2.4.2 Content integrity

Integrity refers to whether documented content is genuinely authored and "real," rather than incomplete, placeholder-based, or residual template material. Integrity defects are common in template-driven documentation and include:

- empty headings (present but without substantive content),
- unfilled placeholders (e.g., `<Name ...>`, `_<... _`),
- `TODO`/`TBD` markers indicating incomplete writing,
- copied template guidance left as section content.

Integrity validation sits between structural and semantic validation: it can be partly rule-based (e.g., placeholder patterns), but it also benefits from contextual interpretation (e.g., distinguishing architecture content from instructions).

2.4.3 Semantic adequacy (fit to intent)

Semantic adequacy evaluates whether the content in a section fulfills the intent of that section as defined by the template guidance. For example:

- a *Runtime View* should describe runtime scenarios and interactions, not merely list components,
- a *Deployment View* should describe environments and mappings of software elements to infrastructure, rather than remaining purely generic,
- a *Design Decisions* section should record decisions and rationale, not only contain headings.

Semantic adequacy is not an absolute judgment of whether the architecture is "good." Instead, it evaluates whether the documentation content matches the architectural purpose of the section. This aligns with the stakeholder/concern basis of architecture descriptions: views exist to address specific concerns, therefore adequacy means addressing those concerns **?**.

2.4.4 Cross-view consistency

Architecture documentation is inherently multi-view. Cross-view consistency is the property that information across views does not contradict and remains coherent. In arc42-derived documentation, typical consistency expectations include:

- elements referenced in runtime scenarios should be defined as building blocks/units in the Building Block View,
- deployment descriptions should account for key building blocks and how they are deployed,
- design decisions should be compatible with stated quality goals (when both are documented),
- risks/technical debt should reflect documented negative consequences or trade-offs of decisions (when present).

Consistency validation therefore checks relationships across sections rather than analyzing sections in isolation.

2.4.5 Traceability vs. consistency

While closely related, traceability and consistency are not identical:

- **Traceability** refers to the existence of links between artifacts or elements (e.g., requirement → architectural decision → component). In documentation auditing, traceability typically means that entities introduced in one location can be followed and located elsewhere, supporting navigation and accountability.
- **Consistency** refers to the absence of contradictions and the presence of coherent alignment across views (e.g., runtime scenarios do not refer to undefined components).

In this thesis, Pass 2 operationalizes an intra-document traceability perspective as a means to detect cross-view consistency issues (i.e., extracting entities and validating consistent references across sections).

2.4.6 False positives and false negatives in documentation auditing

Any automated validation method can produce classification errors:

- a **false positive** flags an issue where none exists (e.g., concise but valid content treated as empty; domain-specific terms mistaken as placeholders),
- a **false negative** misses a real issue (e.g., subtle guidance mismatch not detected; inconsistency implied but not explicitly stated).

In industrial settings, false positives are particularly costly because they reduce trust and adoption. This motivates a conservative staged approach and evidence-backed reporting, even if some subtle issues remain undetected.

2.4.7 Evidence-backed validation and auditability

A central requirement of this thesis is that validation results remain auditable. An output is considered auditable if:

1. every finding is grounded in the document content (no external assumptions),
2. evidence snippets allow a reviewer to verify the claim quickly,
3. the output is structured and reproducible, enabling aggregation and comparison across documents and runs.

This motivates structured JSON outputs with evidence fields and stage-wise intermediate artifacts (Sections 5–6).

3 Literature Analysis

3.1 Structural validation approaches for architecture documentation

A large body of industrial practice addresses documentation quality through template-based standardization. arc42 is one prominent example: it defines a fixed structure and provides guidance for what each section should contain, aiming to make architecture knowledge findable and reviewable **?**. In parallel, architecture description standards emphasize that documentation should be organized around stakeholders and concerns, typically realized via multiple views and viewpoints **?**. These foundations motivate structural validation as an initial quality gate: if required views or sections are missing, the documentation cannot satisfy its intended communication purpose.

Deterministic structural validation typically checks:

- presence or absence of required sections and mandatory headings,
- hierarchy validity (e.g., section–subsection nesting),
- completeness indicators (e.g., empty sections; missing tables/diagrams where expected),
- conformance to organization-specific rules (e.g., mandatory content for compliance).

The strength of this approach is reproducibility and explainability: findings can be traced directly to missing or extra elements. Its limitation is that structural conformance can be misleading: documents may include the right headings but contain placeholders, copied template guidance, or `TODO` markers, creating *false completeness*.

Research on architecture documentation quality and consistency supports why structural checks alone are insufficient. Surveys on architectural inconsistencies report that architecture descriptions are not always trusted or used as intended, and that keeping documentation consistent over time is challenging **?**. This aligns with the industrial constraint that comprehensive manual review does not scale, while overly strict automated checks can create resistance if they generate many false positives.

Implication for this thesis. Structural validation is necessary as a baseline (high precision, explainable), but must be complemented by integrity and semantic checks that detect "structure without substance."

3.2 Similarity and IR-based approaches (alignment, traceability, and reuse detection)

When documentation headings diverge from a template (renames, merged sections, variant terminology), strict structural matchers can over-report missing content. A common way to handle this is to apply Information Retrieval (IR) techniques to find likely correspondences between artifacts (e.g., section titles, headings, paragraphs), treating the problem as a ranking and matching task.

IR-based traceability and alignment has a long history in software engineering. Systematic mappings of IR-based trace recovery show that many approaches rely on algebraic IR models (e.g., TF-IDF, LSI) and similarity measures to propose candidate links, often emphasizing that evaluation and strength-of-evidence varies widely across studies **??**. These approaches are attractive in industrial settings because they:

- require no labeled training data,
- can be incremental and configurable,
- provide explainable similarity scores for candidate correspondences.

Classic work on traceability recovery demonstrates the feasibility of using IR (e.g., LSI) to recover links between artifacts and highlights the need for incremental and maintainable approaches **?**. More recent work continues to treat IR as a practical baseline for recovering links between natural language artifacts and technical artifacts, emphasizing feasibility and usefulness **?**.

For architecture documentation validation, these ideas translate into two practical uses:

1. **Heading/section alignment (template ↔ document):** TF-IDF and cosine similarity can propose that headings such as "System Context" and "Scope and Context" likely correspond, reducing false missing-section signals without requiring exact naming matches.
2. **Boilerplate/template-remnant detection:** Very high similarity between document content and template guidance can indicate copy-paste of instructions instead of authored architecture content.

However, IR similarity is not equivalent to semantic adequacy. IR-based approaches can propose candidate correspondences, but they do not reliably decide whether content actually fulfills architectural intent—especially for sections such as runtime scenarios, deployment mapping, or design rationale.

Implication for this thesis. IR methods are well-supported as alignment and candidate-generation tools, which justifies using TF-IDF/cosine to rescue renamed headings and to flag near-verbatim template reuse, while leaving semantic adequacy to a later stage.

3.3 Machine learning and anomaly-detection approaches (and why they are risky here)

A natural idea for large-scale quality checks is to treat "bad documentation" as an anomaly and use machine learning to detect it. In anomaly detection, one-class classification techniques model "normal" data and flag deviations; these methods have mature ML foundations **?**. In software engineering, anomaly detection is most commonly applied to runtime signals (logs, performance telemetry, failures), where the data is high-volume and the concept of deviation can be statistically defined **?**.

This differs from architecture documentation, where:

- the data is sparse and heterogeneous (prose, lists, tables, diagrams),
- "normal" varies substantially across teams and projects,
- ground truth labels for non-compliance are expensive and subjective,
- interpretability is critical for adoption (engineers need actionable reasons, not only a score).

Even when ML models can detect deviations, they often provide weaker explanations than rule-based missing-structure findings or evidence-backed semantic audits. This aligns with the practical risk observed in exploratory phases: a classifier may learn superficial document traits rather than normative compliance criteria.

Implication for this thesis. ML/anomaly-detection approaches are theoretically relevant, but they introduce high implementation and validation risk in this context (data needs, explainability, variability). This supports a staged design where deterministic checks and IR alignment provide stable coverage first, and semantic reasoning is constrained to produce evidence-backed findings.

3.4 AI and LLMs in software engineering and documentation analysis

Large Language Models (LLMs) have rapidly become prominent in software engineering research and practice, with systematic studies cataloging applications, benefits, and limitations across SE tasks **??**. These studies emphasize typical strategies (prompting, data curation, evaluation) as well as common risks (hallucination, reliability concerns, sensitivity to prompts).

For documentation-centric tasks (including requirements and traceability), the literature increasingly explores LLMs for:

- extraction and classification,
- semantic comparison and validation,
- traceability link recovery (often enhanced via retrieval).

In requirements engineering, recent work highlights that LLMs can support elicitation and validation but must be integrated carefully to address trustworthiness and correctness concerns, often advocating structured processes and safeguards rather than unconstrained generation **?**.

In architecture documentation, a relevant line of work focuses on consistency between natural-language architecture documents and architecture models. The ArDoCo project proposes detecting inconsistencies by establishing traceability between natural-language architecture documentation and architecture models, flagging unmentioned and missing model elements **?**. This is powerful when formal models exist, but many industrial settings rely on text-centric documentation without corresponding formal models. In such cases, the opportunity shifts toward intra-document consistency checks across arc42 views (building blocks ↔ runtime ↔ deployment ↔ decisions/risks), using entity extraction and evidence-based reasoning.

Implication for this thesis. LLMs are well-motivated for semantic adequacy and consistency auditing, but the literature strongly suggests integrating them with constrained outputs (schemas), grounding/evidence requirements, and hybrid pipelines that retain deterministic and IR-based stages for stability **??**. This supports the design choice in this thesis: stabilize structure and alignment via deterministic and IR stages, then apply LLMs in a bounded auditor role that reports only evidence-backed findings.

Transition to research gap. The reviewed approaches each address only parts of the documentation quality dimensions defined in Section 2.3. Structural validators provide deterministic coverage but cannot assess semantic adequacy. IR/similarity methods improve robustness to naming variation and can flag near-verbatim template reuse, but they do not establish that content fulfills architectural intent. ML/anomaly detection may detect statistical deviations, yet it lacks a normative baseline and typically provides insufficient audit-grade explanations. LLMs enable semantic interpretation, but require strict constraints and evidence grounding to be usable in industrial validation workflows. These limitations motivate the integrated, staged validation pipeline proposed in this thesis, formalized in the research gap (Section 3.5) and positioning (Section 3.6).

3.5 Research Gap

Section 2.3 defined architecture documentation quality in this thesis along four operational dimensions: structural completeness, content integrity, semantic adequacy to section intent, and cross-view consistency. When mapped to existing research and industrial practice, a clear pattern emerges: most approaches address only a subset of these dimensions and often rely on assumptions that do not hold in text-centric industrial documentation settings.

Structural validation is necessary but shallow. Template-based frameworks such as arc42 provide a normative structure and guidance for what should be documented and where (Section 2.2). This enables deterministic validation of structural completeness (presence of required sections and expected hierarchy). However, structural conformance alone is insufficient: a section can exist while containing placeholders, residual template instructions, or generic text, creating *false completeness*. This limitation is consistent with empirical observations that architecture descriptions may exist yet lose practical value as they diverge from evolving systems and stakeholder needs **?**.

Documentation debt and evolving inconsistencies motivate validation beyond presence checks. Empirical work reports that architectural inconsistencies evolve over time and that architecture descriptions are not always trusted or used to the extent intended **?**. In parallel, research on documentation technical debt characterizes recurring documentation deficiencies (e.g., non-existent, inadequate, or incomplete documentation) and their persistence under organizational constraints **?**. Together, these findings motivate validation that targets integrity ("is it genuinely authored and informative?") rather than only structural presence.

Consistency and traceability research often assumes formal models that are unavailable in many contexts. Several architecture-consistency approaches detect mismatches by recovering links between informal text and formal architecture representations. ArDoCo, for instance, detects inconsistencies between natural-language software architecture documentation and architecture models via traceability link recovery and identifies unmentioned model elements and missing model elements **?**. While this validates the general principle that inconsistency detection via recovered relationships is feasible and valuable, it typically presupposes the existence of formal models and a supporting tooling ecosystem. Many organizations, however, maintain primarily text-centric arc42(-derived) documentation without corresponding formal architecture models, which limits direct applicability of model-based approaches.

Similarity/IR techniques help alignment but do not establish semantic correctness. Information retrieval methods (e.g., TF-IDF/cosine similarity) are widely used to propose candidate correspondences between textual artifacts (e.g., for traceability link recovery). For arc42 documentation, such techniques can compensate for naming variation (e.g., renamed headings) and can also flag near-verbatim reuse of template guidance. However, lexical similarity is not equivalent to semantic adequacy: IR methods can propose candidates but cannot reliably assess whether section content fulfills its architectural intent, especially for behavioral and rationale-centric sections.

LLMs provide semantic capability but raise reliability and auditability concerns. Recent surveys of LLMs in software engineering document broad applicability but also emphasize recurring risks such as hallucination, sensitivity to prompts, and the need for careful evaluation and constraints **?**. For documentation auditing, the challenge is therefore not only whether LLMs can interpret content, but how to constrain them to produce reliable, verifiable, and audit-grade findings suitable for industrial review.

Research gap. Taken together, the literature motivates a gap that directly matches the industrial problem addressed by this thesis:

> There is a lack of an integrated, audit-oriented validation approach that operates on text-only arc42 (or arc42-derived) architecture documentation, combines deterministic structural guarantees with robustness to naming variance, performs integrity and semantic adequacy checks grounded in section intent, and validates cross-view consistency through explicitly reported, evidence-backed findings.

This gap is particularly relevant in industrial environments where architecture documentation exists primarily in textual form and must be reviewed under time constraints, making explainability and actionable outputs essential.

3.6 Positioning of This Work

This thesis positions its contribution as an engineering synthesis of complementary techniques—deterministic structural validation, similarity-based alignment, and constrained LLM-based semantic auditing—to address the four quality dimensions defined in Section 2.3 within the constraints of text-centric, arc42-derived documentation.

Normative backbone: arc42(-derived) reference template. arc42 provides a structured, multi-view documentation model with explicit section intent and guidance (Section 2.2), enabling a normative definition of what *should* be documented and where. In this thesis, validation is performed against the company-specific arc42-derived reference template used in the case-study environment, while arc42 remains the conceptual foundation. This grounds validation criteria in an established documentation framework rather than subjective judgments.

Pipeline design: from deterministic guarantees to constrained semantics. The approach is implemented as a staged pipeline in which each stage contributes distinct capabilities and controls specific failure modes:

1. **Baseline structural validation (deterministic).** Validates structural completeness by checking required sections/headings and hierarchy against the reference template. This stage provides a stable and reproducible contract check that does not depend on probabilistic inference.
2. **Similarity-based resolver (robust alignment).** Uses IR-style similarity to rescue matches when headings deviate from the template (e.g., renames), improving robustness without claiming semantic correctness. Similarity is used as a controlled alignment mechanism rather than a proxy for documentation quality.
3. **Preprocessing and normalization (audit-ready representation).** Produces a normalized representation that couples each section with its guidance intent, enabling controlled downstream reasoning. This directly supports auditable semantic validation by constraining what is assessed (section content) against what is expected (template guidance).
4. **Pass 1 LLM audit: integrity and semantic adequacy (bounded).** Applies an LLM to detect stubs/placeholders and evaluate whether content fulfills the intent implied by the section guidance. The motivation is grounded in empirical observations that architecture documentation degrades over time and loses trust when inconsistencies evolve **?**, and in documented persistence of documentation deficiencies as technical debt **?**. To mitigate reliability concerns, the audit is constrained through structured inputs and evidence-backed, schema-validated outputs **?**.
5. **Pass 2 consistency audit: cross-view coherence via intra-document traceability.** Implements cross-view consistency checks across arc42 views by extracting architectural entities and validating their coherent references across sections. The design is inspired by the general principle in architecture consistency research that mismatches can be detected via recovered relationships across representations **?**, but it remains applicable to text-only documentation by reconstructing relationships directly from the document.

Contribution relative to existing work. Compared to purely structural validators, this work extends validation into integrity and semantic adequacy. Compared to IR-only approaches, it uses similarity as a robustness mechanism for alignment and boilerplate detection rather than a substitute for meaning. Compared to model-based consistency frameworks such as ArDoCo, it targets the common industrial scenario of text-only architecture documentation while retaining the core idea of detecting inconsistencies via reconstructable relationships. Finally, compared to unconstrained LLM usage, this work follows recommendations highlighted in LLM4SE surveys by embedding LLM reasoning into a structured, evidence-grounded validation pipeline rather than treating it as an open-ended generator **?**.

Expected impact. By turning architecture documentation validation into a staged, auditable process, the approach aims to increase the detectability and localization of documentation defects (Section 2.3.6). This supports industrial review workflows where trust, traceability of findings, and actionable outputs are critical, and where manual validation alone does not scale.

4 Industrial Context and Requirements

This section describes the industrial environment in which the proposed validation approach is developed, the current documentation workflow and limitations observed in practice, and the resulting requirements that shape the design of the system. The goal is to make explicit the constraints and motivations that led to the staged pipeline architecture chosen in this thesis.

4.1 Current practice and tooling

In the industrial context of this work, software architecture documentation is authored primarily in AsciiDoc (`.adoc`) using an arc42-derived company template (Section 2.2). Architecture documents are maintained per software unit or subsystem (e.g., SWE-level documentation) within internal repositories.

A central automation component in the current workflow is a crawler tool that processes these `.adoc` documents at scale. The crawler parses architecture documentation files, generates PDF outputs, and produces statistics about detected issues per SWE. These outputs provide a high-level overview of documentation status across multiple units and support quality reporting.

Despite this automation, the validation scope in practice is limited. Based on discussions with quality stakeholders (including quality management and architecture stakeholders), existing automated checks were intentionally restricted to a small subset of chapters (notably including Design Decisions ..) .This reflects a trade-off between coverage and acceptance: early automation prioritizes low-noise checks over comprehensive validation. The rationale was pragmatic: enforcing strict checks across all arc42 sections was perceived as likely to produce widespread rejections due to high variability in real documents, creating an excessive burden for teams and undermining acceptance. As a consequence, broader validation is largely conducted through manual review, and in many situations documentation is effectively trusted by default even when quality is uncertain.

Overall, documentation is systematically collected and rendered (AsciiDoc $\rightarrow$ PDF), but comprehensive quality assurance remains either manual or intentionally scoped narrowly to avoid high false-positive rates and resistance from teams.

4.2 Observed challenges and constraints

Meeting notes and stakeholder discussions highlight several recurring challenges that directly influence tool requirements and design:

1. **High variability of documentation content.** Although arc42 provides a stable structure, teams vary widely in how they fill sections: level of detail, naming conventions, granularity, and terminology differ across units. A strict one-size-fits-all validation applied uniformly to all sections can therefore label many documents as non-compliant, even when they are locally considered acceptable. This risk contributed to the earlier decision to restrict automated checks to only a small subset of sections.
2. **Risk of excessive rejection and organizational burden.** Stakeholders expressed that enabling validation for every section could cause most documents to be flagged, leading to continuous rework demands and reduced tool acceptance. This creates a practical constraint: validation must be accurate and explainable enough to avoid being perceived as noise. A tool that produces many false positives becomes counterproductive because teams will ignore it or circumvent it.

3. **Reliance on manual review and implicit trust.** Manual review processes exist, but feasibility is limited across many SWEs and under time pressure. In practice, architecture documentation is often trusted without deep verification, which aligns with the broader phenomenon of documentation debt: shortcomings persist when validation is costly or unclear, and problems become visible only late in development or maintenance.
4. **Need for traceability-related artifacts (e.g., unit mapping).** Stakeholder notes indicate an emphasis on unit mapping and on verifying certain presence relationships. While the exact artifact and rationale must be confirmed, the underlying requirement is clear: architecture documentation should remain connected to technical structuring artifacts that establish traceability of architectural units and responsibilities. This aligns with cross-view consistency expectations (Section 2.3), where building blocks and units should be representable and traceable across views and supporting artifacts.
5. **Workflow integration constraints.** The crawler currently evaluates documents after they exist in repositories, producing PDFs and statistics. A natural extension—raised during this work—is whether validation could act earlier as a quality gate (e.g., prior to uploading or merging), preventing low-quality documentation from entering the repository. This introduces a key practical requirement: validation must be automatable, stable, and produce results that developers can act on quickly.

These constraints collectively motivate the design principles behind the proposed solution: staged validation, explainable outputs, and a balance between strictness and robustness to avoid rejecting documents purely due to naming or structural variation.

4.3 Functional requirements

Based on the current workflow and constraints, the system developed in this thesis is designed to satisfy the following functional requirements:

FR1 — Template-based structural validation. The system shall validate an architecture document against the company's arc42-derived reference template, detecting missing required sections/headings and unexpected extras.

FR2 — Robust alignment under naming variation. The system shall support renamed or slightly altered headings by proposing alignments between template sections and document headings, reducing false missing-section detections caused by naming differences.

FR3 — Integrity validation of content. The system shall detect incomplete documentation content such as empty headings, placeholders, and `TODO`/`TBD`-style stubs, including template remnants that create a false impression of completeness.

FR4 — Semantic adequacy validation against section intent. The system shall evaluate whether the content in each section fulfills the expected architectural intent implied by the reference template guidance, and flag cases where content is irrelevant, boilerplate, or does not address the concern.

FR5 — Cross-section consistency validation. The system shall extract key architectural entities (e.g., units/building blocks, interfaces, external systems, stakeholders when present) and validate their consistent usage across relevant arc42 views, such as:

- entities used in runtime scenarios should correspond to defined building blocks,
- deployment descriptions should account for key building blocks (mapping),
- decisions should be consistent with stated goals (where documented),
- risks should reflect negative consequences of decisions (where available).

FR6 — Evidence-backed findings. For every reported issue, the system shall provide evidence in the form of a relevant text snippet and location context (section/heading), so that reviewers can verify findings quickly.

FR7 — Machine-readable audit output. The system shall produce a structured output (JSON) that can be consumed by downstream tools (dashboards, reports, CI pipelines) and aggregated across multiple SWEs.

FR8 — Batch processing across multiple documents. The system shall support execution across many SWE documents (crawler-style processing) and produce per-document and aggregated results (e.g., error statistics per SWE).

4.4 Non-functional requirements

To be viable in the given industrial workflow—especially if used as a quality gate—the following non-functional requirements are central:

NFR1 — Explainability and auditability. Outputs must be interpretable by engineers and quality stakeholders. Findings must be traceable to evidence rather than opaque scores, supporting trust and adoption.

NFR2 — Robustness with controlled strictness. Validation must tolerate reasonable variation (e.g., renamed headings) while remaining strict enough to detect real quality issues. The tool should minimize false positives to avoid overwhelming teams.

NFR3 — Reproducibility and determinism where possible. Repeated runs on the same input should yield consistent results, particularly for structural checks and integrity checks. Where probabilistic methods are used, outputs should be constrained and structured.

NFR4 — Scalability to organizational usage. The tool should be efficient enough to run across many SWE documents as part of the existing crawler process, enabling organization-wide monitoring rather than single-document analysis only.

NFR5 — Integration friendliness. The system should integrate into existing pipelines (e.g., crawler or pre-merge quality gate) with minimal friction: simple input formats (AsciiDoc/JSON), machine-readable output, and clear error reporting.

5 Methodology

This section describes the research and engineering methodology used to design, implement, and evaluate the proposed approach for AI-based structural and semantic validation of architecture documentation. The methodology is derived from the problem framing and quality dimensions introduced in Section 2, the research gap and positioning established in Section 3, and the industrial context and requirements captured in Section 4. The central methodological objective is to construct a validation pipeline that is (i) grounded in a normative documentation framework (an arc42-derived reference template), (ii) robust to real-world variation, and (iii) capable of producing auditable, evidence-backed findings suitable for industrial use.

5.1 Research design and approach

The work follows an engineering-oriented research design: it develops a practical artifact (a validation toolchain) and evaluates its usefulness in an industrially relevant setting. The thesis is not positioned as a purely theoretical contribution, but as an applied approach that operationalizes documentation quality dimensions (Section 2.3) into concrete checks and outputs.

Methodologically, the thesis follows a design-and-evaluate cycle:

1. problem analysis and requirement elicitation from the industrial workflow (Section 4),
2. derivation of validation objectives from documentation quality dimensions and arc42 section intent (Sections 2.2–2.3),
3. pipeline design that decomposes the validation problem into deterministic, similarity-based, and semantic reasoning stages,
4. modular implementation of the pipeline,
5. evaluation on real architecture documentation to assess defect detection coverage, usefulness, and explainability (Section 7).

This structure ensures that each technical choice is traceable to either the reference framework (arc42-derived template), an explicit quality dimension, or an industrial constraint (e.g., variability and false-positive risk).

5.2 Inputs, artifacts, and scope of validation

The validation pipeline operates on the following primary inputs:

- **Reference template:** the company-specific arc42-derived template used as the normative baseline for structural validation and as the source of section-intent guidance.
- **Architecture documentation:** AsciiDoc (`.adoc`) documents authored per software unit/subsystem and processed in the industrial environment (e.g., at SWE level).
- **Parsed representation:** a normalized intermediate representation (JSON) derived from parsing the AsciiDoc files into a section/heading structure and associated content blocks.

The scope of validation in this thesis is text-centric architecture documentation. The approach does not assume the presence of formal architecture models (e.g., UML, PCM) or code-level ground truth. Consistency checking is therefore implemented as *intra-document architectural traceability* across arc42 views (Section 2.3.5), rather than cross-artifact traceability (e.g., documentation-to-code). Where industrial artifacts such as unit mappings exist (e.g., a unit mapping file), these are treated as optional integration points and discussed as extensions in Section 8.

5.3 Quality model operationalization

Section 2.3 defined documentation quality along four dimensions: structural completeness, integrity, semantic adequacy, and cross-view consistency. This thesis operationalizes these dimensions into staged validation responsibilities:

- **Structural completeness** → deterministic checks against the reference template structure.
- **Integrity** → detection of incomplete/stub/placeholder content that can exist even when structure is present.
- **Semantic adequacy** → assessment of whether section content fulfills the intent expressed in the template guidance.
- **Cross-view consistency** → detection of contradictions or missing relationships across building blocks, runtime scenarios, deployment mappings, decisions, goals, and risks.

The central methodological principle is that these dimensions require different techniques. Structural completeness can be assessed deterministically, whereas semantic adequacy requires interpretation. This motivates a hybrid pipeline combining deterministic checks, similarity-based alignment, and constrained semantic auditing.

5.4 Staged validation pipeline

To balance determinism, robustness, and semantic depth, validation is implemented as a multi-stage pipeline. Each stage contributes a specific capability and is designed to control a specific class of failure modes.

5.4.1 Stage 0: Parsing and normalization boundary

Before validation, architecture documentation is parsed from AsciiDoc into a structured representation. This establishes a stable boundary between authoring formats and validation logic. The output represents sections and headings, hierarchical levels, and associated content blocks. This normalization supports reproducibility and makes subsequent reasoning explicit and traceable.

5.4.2 Stage 1: Baseline structural validation (deterministic)

The baseline checker validates the document structure against the reference template: required sections/headings are present, hierarchy matches the expected structure, and unmatched/extra headings are identified. This stage targets the structural completeness dimension and provides deterministic coverage of missing structure. It intentionally does not judge content adequacy.

5.4.3 Stage 2: Similarity-based alignment for naming variation

Real-world documents often rename or slightly modify headings. To reduce false missing-section findings caused by naming variance, the pipeline includes a similarity-based resolver that proposes alignments between unmatched document headings and template headings (candidate matching). Similarity is used for alignment, not for quality scoring. The output remains a mapping of likely correspondences that is subsequently validated by semantic checks.

5.4.4 Stage 3: Preprocessing for auditable semantic analysis

The pipeline produces a final preprocessed JSON representation that consolidates resolved heading alignment, extracted guidance per section, normalized content, and integrity signals. This ensures that semantic auditing consumes a stable, bounded input and supports strict output constraints.

5.4.5 Pass 1 audit: integrity and semantic adequacy (section-local)

Pass 1 performs section-local analysis. For each arc42 section and its headings, it identifies integrity defects (stubs, placeholders, empty headings, template remnants) and semantic mismatches (content does not satisfy the section guidance intent). Boilerplate or irrelevant content patterns are flagged where supported by evidence.

The core method for Pass 1 is constrained semantic reasoning with a large language model. The model is treated as an auditor rather than a generator: outputs are restricted to a structured schema, and every reported issue must include an evidence snippet from the input. Pass 1 primarily targets the integrity and semantic adequacy dimensions.

5.4.6 Pass 2 audit: cross-view consistency (global)

Pass 2 performs global analysis across sections and views by extracting and comparing architectural entities and their relationships. It evaluates cross-view consistency expectations such as: runtime scenarios referencing defined building blocks, deployment sections accounting for building blocks and their mappings, decisions relating to documented goals, and risks reflecting negative consequences or trade-offs where present.

Pass 2 operationalizes cross-view consistency as intra-document traceability checks: entities introduced in one view should be used coherently in others. Outputs are again evidence-based and structured.

5.4.7 Output model and auditability constraints

Both passes produce results as structured JSON suitable for aggregation in the crawler context. The methodology enforces two auditability principles:

1. **Evidence-backed reporting:** every finding must include a text snippet supporting the claim.
2. **Bounded outputs:** the auditor may only report what is grounded in the provided content; unsupported claims are not permitted.

These constraints enable practical review and reduce hallucination risk.

5.5 Scoring and interpretation model

To summarize findings for reporting and prioritization, each arc42 section is assigned an ordinal score (0–5). The score is not intended to be a measure of architectural quality in an absolute sense; instead, it summarizes detected documentation issues relative to the quality dimensions.

A typical interpretation is:

- **5:** content fulfills intent; no integrity or semantic defects detected,
- **4:** minor issues; section largely complete and adequate,
- **3:** partial adequacy; missing detail or isolated integrity issues,
- **2:** significant gaps; multiple integrity/adequacy issues,

- **1:** mostly missing/placeholder/boilerplate; low usefulness,
- **0:** section absent when required by the reference template.

The rubric is applied consistently across documents and used in the evaluation to compare stages and identify where defects concentrate (e.g., integrity-heavy vs. semantic-heavy vs. consistency-heavy issues).

5.6 Evaluation plan (method-level overview)

The evaluation (Section 7) follows directly from the quality dimensions and pipeline structure. The approach is evaluated through:

- **Defect detection coverage:** how many and which types of issues are detected at each stage,
- **Stage contribution analysis:** what additional issues are found when adding similarity resolution, Pass 1 semantic auditing, and Pass 2 cross-view checks,
- **Usefulness and explainability:** whether findings are actionable and verifiable via evidence snippets.

Where possible, findings are compared against manual review outcomes and practitioner expectations in the industrial context. The evaluation emphasizes practical value: a validation method is considered successful if it improves detectability and localization of documentation defects while remaining auditable and manageable for teams.

6 System Architecture and Implementation

The implementation of the staged validation pipeline introduced in Section 5 is organized into modular components that correspond to the pipeline stages (baseline structural validation, similarity-based alignment, preprocessing for semantic auditing, and the audit passes). The design goal of the implementation is twofold: (i) to keep deterministic stages fully reproducible and explainable, and (ii) to provide the audit passes with an auditable, bounded input representation that enables evidence-backed findings.

6.1 Implementation overview and module structure

The implementation is structured around three core Python modules that realize Stages 0–3 of the pipeline:

- `baseline_check.py`: implements Stage 1 deterministic validation against the reference template (structural completeness and integrity signals such as empty/stub detection).
- `cosine_tfidf_layer.py`: implements Stage 2 similarity-based alignment and additional integrity heuristics (rename rescue, placeholder detection via similarity, template-content detection, and duplicate-content detection).
- `llm_preprocess.py`: implements Stage 3 preprocessing, consolidating deterministic and similarity findings into a single normalized JSON payload with guidance attached, ready for Pass 1 and Pass 2 auditing.

The pipeline produces layered intermediate artifacts per document, enabling stage-wise inspection and evaluation:

1. `baseline_report_<doc_id>.json`
2. `cosine_resolved_report_<doc_id>.json`
3. `preprocessed_<doc_id>.json`

This layered output supports debugging and enables the stage-contribution analysis reported in Section 7.

6.2 Data model and intermediate representations

All stages operate on a consistent JSON structure representing parsed AsciiDoc documents. In the parsed representation (e.g., `car_siriussxm360.json`), each document contains a list of sections; each section contains headings; each heading contains:

- `text` (heading title),
- `level` (heading level; e.g., 2 for top-level arc42 chapters),
- `type` (static/placeholder/other classification from parsing),
- `raw_text` (content associated with the heading),
- optional artifact flags (e.g., `has_table`, `has_diagram`).

The reference template is represented in `expected_spec.json`. It defines expected top-level sections and headings, their types (static vs. placeholder), requiredness, and the guidance text later used for semantic adequacy checks. By representing both template and document in the same structural model, matching and guidance attachment remain deterministic and explainable.

6.3 Stage 1: Baseline structural validation (`baseline_check.py`)

The baseline checker implements the deterministic validation described in Section 5.4.2. A key design choice is that this stage remains normative and reproducible, avoiding probabilistic decisions.

6.3.1 Normalization strategy

Section and heading matching is performed using strict normalization:

- trimming whitespace,
- removing trailing heading markers and syntax noise,
- lowercasing titles.

This ensures that superficial formatting differences do not produce false "missing" results.

6.3.2 Mandatory section and static heading checks

The validator loads the reference template (`expected_spec.json`) and identifies mandatory Level-2 sections (arc42 main chapters). For each expected section, it checks whether the parsed document contains a matching section title. For matched sections, it checks the presence of required static sub-headings and reports:

- missing mandatory sections,
- missing static headings,
- extra sections,
- unmatched sample headings.

The result is returned in a report containing a baseline summary, structural gaps, and an audit trail for traceability.

6.3.3 Integrity signal: "empty heading" detection

Although Stage 1 is primarily structural, it additionally computes an integrity signal indicating whether a heading is effectively empty. The `_is_empty()` function treats a heading as non-empty if it includes engineering artifacts (diagram/table flags). Otherwise, it strips AsciiDoc syntax and removes recognized placeholder patterns (e.g., `<...>`, `_ <...>_`) and common stub markers (e.g., `TODO`, `TBD`). A heading is flagged as effectively empty when the remaining content falls below a minimal-length threshold. This check is intentionally conservative: it does not claim semantic adequacy, but flags content likely to be incomplete.

6.3.4 Explainability: audit trail

The baseline report includes an `audit_trail` per matched section, listing each heading with normalized matching status (e.g., static match vs. placeholder vs. extra), emptiness signals, and artifact flags. This audit trail is reused in preprocessing to attach deterministic status to each heading in the final audit package.

6.4 Stage 2: Similarity-based alignment and integrity heuristics (`cosine_tfidf_layer.py`)

The cosine similarity layer implements Stage 2 from Section 5.4.3. Its purpose is to improve robustness under real-world naming variation while keeping decisions inspectable.

6.4.1 Vectorization choice

The implementation uses a TF-IDF vectorizer configured for character n-grams (1–3). Character n-grams are more robust than word-level tokens for minor spelling variation, compound words, and naming differences common in heading titles. The vectorizer is fit on template-derived texts, including section titles, placeholder headings, and guidance texts extracted from the reference template, thereby creating a stable comparison space.

6.4.2 Section rescue mapping for renamed headings

To reduce false "missing mandatory section" reports, the resolver matches missing template sections to extra sections found in the document. It computes cosine similarity between missing template section titles and extra document section titles and, when similarity exceeds a defined threshold, records a rescue mapping (document title → template title) with a similarity score. The mapping is treated as an alignment hypothesis used later in preprocessing rather than as a quality score.

6.4.3 Similarity-based integrity checks

In addition to rename rescue, the module implements three integrity-related heuristics:

1. **Unfilled placeholder heading detection.** Placeholder headings from the template are embedded and compared to document heading titles. If similarity exceeds a high threshold, the heading is flagged as an unfilled placeholder, capturing cases where template slots were never replaced.
2. **Template guidance left in content (copy/paste detection).** For headings with sufficient body text, the resolver compares heading content to the corresponding template guidance. If similarity is extremely high, the content is flagged as copied template guidance. This rule is conservative and triggers only under near-verbatim similarity.
3. **Duplicate content across headings.** A cosine similarity matrix is computed across longer prose blocks. Pairs above a high similarity threshold are flagged as potential duplicates, supporting copy/paste detection across unrelated sections.

6.4.4 Special-case handling: legal note

The implementation includes a targeted rule for the legal note, where reuse of template license text is expected. If similarity to the template license content falls below a threshold, a warning is emitted. This illustrates how the framework can support section-specific policies where template reuse is intended and appropriate.

6.4.5 Output

The cosine layer outputs a summary (counts of rescued sections, integrity violations, duplicates) and a detailed analysis including rescue mappings and integrity/duplicate violations.

6.5 Stage 3: Preprocessing for audit-ready input (`llm_preprocess.py`)

The preprocessing layer implements Stage 3 from Section 5.4.4. Its objective is to unify earlier findings into a single representation that is (i) stable, (ii) bounded, and (iii) guidance-aware.

6.5.1 Consolidation of signals

The preprocessor loads the reference template, the parsed document, the baseline report, and the cosine report. It constructs lookups for integrity violations keyed by section/heading and for rescue mappings keyed by template section titles. Baseline deterministic statuses are also attached per heading to preserve explainability.

6.5.2 Guidance attachment strategy

For each template section, the preprocessor includes section-level guidance and a list of headings to audit. Guidance is attached hierarchically:

1. heading-specific guidance from the template (if available),
2. otherwise section-level guidance,
3. otherwise a generic fallback guidance string.

This ensures that every heading has an explicit "expected intent" field, enabling later semantic adequacy evaluation.

6.5.3 Final audit package format

The resulting JSON contains `document_metadata` and a `sections_to_audit` array. For each template section it includes presence status (`FOUND`/`MISSING`), guidance, and a list of headings containing:

- heading text and level,
- baseline structural status (e.g., static match, extra, placeholder),
- integrity status (clean or violation type),
- guidance (intent),
- content (raw text).

This file (`preprocessed_<doc_id>.json`) is the contract between deterministic preprocessing and downstream auditing and is the only required input for Pass 1 and Pass 2.

6.5.4 Batch execution

The module includes batch execution support to generate preprocessed inputs for all documents for which baseline and cosine artifacts exist, enabling crawler-style processing across many SWEs.

6.6 Pass 1 implementation: integrity and semantic adequacy audit

Pass 1 corresponds to Section 5.4.5 and performs section-local auditing. It consumes `preprocessed_<doc_id>.json` and produces a structured audit report designed to be auditable and machine-readable.

6.6.1 Bounded input packaging and context control

The input to Pass 1 is restricted to the aligned sections and headings provided in the preprocessed representation. For each heading, the payload includes: (i) guidance text describing the intent of the section/heading, (ii) deterministic and similarity-derived integrity signals, and (iii) the actual heading content. To keep behavior consistent across documents of different size, a deterministic content budget is applied per heading (e.g., truncation to a fixed character or token limit). The truncation strategy preserves the beginning of content blocks and retains short structured lines (lists, interface enumerations) where possible, since these frequently contain evidence relevant for auditing.

6.6.2 Prompt management as a versioned artifact

Pass 1 instructions are maintained as a versioned prompt file (e.g., `prompts/pass1_system.txt`). Treating prompts as versioned artifacts is essential for reproducibility and evaluation: changes to the prompt can systematically change outputs. The prompt enforces the following constraints:

- report only findings that are supported by evidence in the provided content,
- distinguish integrity issues (e.g., stubs, placeholders) from semantic issues (guidance mismatch),
- do not rewrite or generate content; only audit what exists,
- keep evidence snippets short and directly grounded.

6.6.3 Schema-constrained output and validation

Pass 1 uses schema-constrained output to enforce machine-readability and prevent uncontrolled prose. Outputs must conform to a JSON schema (e.g., `schemas/pass1_schema.json`) validated at runtime. Schema enforcement prevents failure modes such as missing required keys, vague findings without evidence, and inconsistent field naming across runs. If schema validation fails, the run is handled via bounded retry logic; persistent failures are recorded as run errors rather than producing partial outputs.

6.6.4 Scoring model and section-level rationale

Each arc42 section receives an ordinal score (0–5) that summarizes the severity and density of detected issues and supports prioritization (Section 5.5). The scoring is accompanied by a short rationale to keep the score reviewable. The rubric is applied consistently:

- 0: section missing when required,
- 1: mostly placeholders/stubs or template remnants; no meaningful content,
- 2: multiple serious integrity issues and/or strong guidance mismatch,
- 3: partially adequate; mixed quality; incomplete but meaningful core content exists,
- 4: minor gaps; mostly adequate,
- 5: adequate relative to guidance; no issues detected.

6.6.5 Pass 1 output artifact

Pass 1 produces `pass1_audit_<doc_id>.json` containing per-section entries with a score, integrity findings (heading-level issue objects with evidence), semantic findings (heading-level issue objects with evidence), a list of affected headings, a primary evidence snippet per section, and the score rationale. This structure supports aggregation across many SWEs while remaining auditable for manual verification.

6.7 Pass 2 implementation: cross-view consistency audit

Pass 2 corresponds to Section 5.4.6 and focuses on cross-view coherence. Unlike Pass 1 (section-local), Pass 2 reconstructs intra-document traceability by extracting architectural entities and verifying their coherent use across arc42 views.

6.7.1 Inputs and reuse of preprocessing

Pass 2 consumes the same preprocessed representation as Pass 1. Optionally, Pass 2 can reuse extracted entities or section adequacy signals from Pass 1 outputs, but its core operation remains grounded in the preprocessed content to avoid dependency on non-deterministic intermediate decisions.

6.7.2 Entity extraction strategy

Entities are extracted and normalized into typed sets, for example:

- **Units/building blocks** (e.g., unit markers, BBV lists, component identifiers),
- **Interfaces** (e.g., HTTP, AIDL, HAL, sockets, APIs),
- **External systems** (e.g., external services, hardware, OS-level actors),
- **Stakeholders** (if present),
- optionally: quality goals and decision statements (if present).

Extraction is implemented in two layers. First, deterministic cues are applied (section-based mining, bullet list extraction, identifier heuristics). Second, when deterministic cues are insufficient, constrained semantic extraction is used to produce structured entity lists from the provided text. Entities are normalized (trim, de-duplicate, safe case normalization) to reduce false mismatches.

6.7.3 Consistency rules

Pass 2 implements conservative rule checks that only report inconsistencies when explicit textual evidence exists:

- **Static vs. dynamic alignment (BBV vs. Runtime).** Entities referenced in runtime scenarios should be defined as building blocks/units in the building block view.
- **Infrastructure mapping (BBV vs. Deployment).** Deployment content should account for key building blocks and their mapping to infrastructure elements/environments, where applicable.
- **Goal alignment (Quality goals vs. Design decisions).** If quality goals are documented, decisions should not contradict them and should reference relevant drivers where possible.
- **Risk consistency (Decisions vs. Risks/technical debt).** If decisions explicitly document negative consequences or trade-offs, the risks/technical debt section should reflect these concerns.

The output remains intentionally conservative: when a relationship cannot be verified from text, it is not reported as an inconsistency.

6.7.4 Pass 2 output artifact

Pass 2 produces `pass2_consistency_<doc_id>.json` containing (i) typed entity sets (`entities_seen`) and (ii) a list of evidence-backed consistency issues. Optional rule-level counters can be included to support evaluation and quantitative comparisons across runs.

6.8 Integration and execution in the industrial workflow

Stages 1–3 (baseline, cosine, preprocessing) are designed to integrate naturally with the current crawler-based workflow (Section 4). They operate on parsed AsciiDoc documents already handled by existing tooling and produce machine-readable JSON outputs suitable for aggregation per SWE. The staged design supports incremental adoption: deterministic stages provide immediate value even before audit passes are enabled.

A natural integration extension is to run the pipeline earlier (e.g., as a documentation quality gate before merging or uploading). The staged design supports this by enabling strict enforcement for deterministic checks while configuring audit passes in reporting-only mode initially to reduce organizational burden.

6.9 Configuration and thresholds

The pipeline uses explicit thresholds and policies to control sensitivity and reduce false positives. Key parameters are intentionally conservative, reflecting the industrial constraints identified in Section 4.

Deterministic integrity threshold (Stage 1). A heading is considered effectively empty when cleaned content length falls below a minimal threshold after stripping syntax and placeholder patterns. Artifact flags (tables/diagrams) override emptiness and are treated as non-empty.

Cosine similarity thresholds (Stage 2). The similarity layer uses thresholds for (i) section rescue mapping, (ii) placeholder detection, (iii) template guidance copy detection, and (iv) duplicate prose detection. Duplicate detection is restricted to sufficiently long prose blocks to prevent spurious matches on short text.

Section-specific policy (legal note). For sections where template reuse is intended (e.g., legal note), a low similarity triggers a warning indicating that expected license text may have been modified.

These values are evaluated in Section 7 through stage contribution analysis and qualitative inspection of false positives.

6.10 Reproducibility, logging, and run metadata

Reproducibility is treated as a first-class implementation goal because the evaluation compares stages and audit outputs across documents.

6.10.1 Intermediate artifacts and traceability

For each input document, the pipeline persists intermediate outputs: baseline report, cosine report, preprocessed audit package, and the audit outputs of Pass 1 and Pass 2. Persisting intermediate artifacts enables (i) debugging, (ii) stage contribution measurement, and (iii) auditability of results by allowing reviewers to trace any finding back to a specific content snippet and upstream preprocessing decision.

6.10.2 Logging and failure handling

Each stage logs processed documents, generated outputs, counts of detected gaps/violations, and warnings when expected artifacts are missing (e.g., missing cosine output causes preprocessing to skip that document). Batch execution uses bounded exception handling to prevent a single bad document from stopping organization-scale processing.

6.10.3 Run metadata

To ensure comparability across experiments, each output artifact includes a `run_metadata` block capturing configuration identifiers (threshold set), prompt and schema versions, and run identifiers. This supports evaluation questions such as whether prompt changes alter findings, whether threshold tuning changes false positives, and whether results remain comparable across runs.

6.11 Implementation summary

The implementation realizes the staged validation methodology as a modular and reproducible toolchain. Deterministic stages establish template-based structural compliance and a stable normalized representation, similarity-based alignment improves robustness to real-world naming variance, and the audit passes operate on guidance-aware inputs with schema-constrained, evidence-backed outputs. Intermediate artifacts are persisted to enable debugging and stage contribution analysis, and configuration/prompt/schema versioning supports controlled experimentation and industrial integration.

7 Evaluation

This chapter evaluates the proposed staged validation pipeline with respect to the documentation quality dimensions defined in Section 2.3 (structural completeness, integrity, semantic adequacy, and cross-view consistency). The evaluation is designed so that the study setup and the analysis protocol remain valid independently of the final numerical outcomes; quantitative result sections are completed once Pass 1 and Pass 2 are executed on the evaluation corpus.

7.1 Evaluation objectives

The evaluation assesses whether the pipeline improves the detection and localization of architecture documentation issues in a way that is auditable and practically useful. Three objectives guide the evaluation:

1. **Stage contribution:** quantify and explain what each stage adds (baseline → cosine resolver → preprocessing → Pass 1 → Pass 2).
2. **Auditability:** verify that outputs remain evidence-backed and actionable, consistent with the auditability constraints in Section 5.4.7.
3. **Industrial relevance:** assess whether detected issues align with reviewer expectations and whether the output format can support real review workflows.

7.2 Case study and dataset setup

7.2.1 Industrial document corpus

The evaluation corpus consists of real architecture documentation authored using an arc42-derived company template and maintained in AsciiDoc (`.adoc`) format. Documents are maintained per software unit/subsystem (e.g., SWE-level documentation) and are processed in batch form, consistent with the workflow described in Chapter 4.

Corpus scope (to fill).

- Number of SWE documents: **[N]**
- Snapshot date / time window: **[DATE]**
- Document selection criterion: **[e.g., latest mainline versions]**
- Template version identifier: **[Template ID / commit hash]**

7.2.2 Reference template

All validation is performed against the company-specific arc42-derived reference template described in Section 2.2. Template guidance is used both as normative expectation for deterministic structure checks and as the reference intent for semantic adequacy checks.

Template source (to fill).

- Template artifact: `expected_spec.json` (derived from company template AsciiDoc)
- Template revision: **[commit hash / version tag]**

7.2.3 Preprocessing and input normalization

Before evaluation, documents are parsed and normalized into the structured JSON representation described in Chapter 6. The representation captures section/heading hierarchy, content blocks, and associated template guidance per section/heading. The evaluation uses persisted intermediate artifacts (baseline reports, cosine reports, and preprocessed inputs) to ensure that stage-by-stage comparisons are reproducible.

7.3 Metrics, criteria, and evaluation protocol

This section defines the evaluation measures and how results are interpreted.

7.3.1 Structural metrics (Stage 1 baseline)

Structural completeness is measured using deterministic signals from the baseline reports:

- Missing required sections (count per document; list of missing sections)
- Missing required headings (count per section; list of headings)
- Hierarchy mismatches (if enabled; count and types)
- Extra sections/headings (count; treated as deviation/extension signals)

7.3.2 Robustness metrics (Stage 2 cosine resolver)

Robustness to naming variation is measured by:

- Rescued mappings count: number of template sections aligned to document sections via similarity
- Reduction in missing-section findings after applying rescue mappings (before → after)
- Precision sanity checks: sampled manual verification of rescued mappings (sampling strategy in Section 7.6)

Additionally, optional indicators from the cosine layer are reported:

- high-confidence unfilled placeholder headings detected,
- near-verbatim template guidance copies detected,
- duplicate content candidates detected.

7.3.3 Integrity and semantic adequacy metrics (Pass 1)

Pass 1 evaluates integrity and semantic adequacy at section/heading granularity. The following measures are reported:

- Integrity issue count per document and per section (e.g., placeholders, TODOs, empty headings)
- Semantic mismatch count per document and per section (guidance not satisfied, irrelevant content, boilerplate)
- Evidence coverage rate: percentage of findings that include a valid evidence snippet
- Section score distribution (0–5) across documents and sections

In addition to quantitative counts, a qualitative inspection assesses whether findings are clearly justified and human-verifiable.

7.3.4 Cross-view consistency metrics (Pass 2)

Pass 2 evaluates cross-view consistency through entity extraction and conservative rule checks. Measures include:

- Entity coverage: counts of extracted entities by type (units/building blocks, interfaces, external systems, stakeholders)
- Consistency issue count by rule category:
  - Building Block View ↔ Runtime View (undefined entities referenced in runtime scenarios)
  - Building Block View ↔ Deployment View (missing or unclear mappings)
  - Goals ↔ Decisions (contradictions or missing alignment where checkable)
  - Decisions ↔ Risks/Technical Debt (unreflected trade-offs where explicit evidence exists)
- Evidence coverage rate for consistency issues

7.3.5 Stage contribution analysis (primary evaluation lens)

The primary evaluation method is a stage comparison:

1. Stage 1 only (baseline)
2. Stage 1 + Stage 2 (baseline + cosine resolver)
3. Stage 1 + Stage 2 + Stage 3 (preprocessed audit readiness)
4. Pass 1 (integrity + semantic adequacy)
5. Pass 2 (cross-view consistency)

For each stage set, the evaluation reports:

- additional issues found,
- issues resolved or reduced (e.g., fewer false "missing" sections after rescue),
- changes in section scores (where applicable),
- representative qualitative examples illustrating the incremental benefit.

7.4 Results on real documents

This section reports quantitative results once the pipeline is executed on the corpus.

7.4.1 Dataset summary

To fill after execution.

- Documents processed successfully: **[N_success]** / **[N_total]**
- Failures and reasons: **[N_fail + reasons]**
- Runtime per document (median/mean): **[time]**
- Output artifact sizes (average): **[avg JSON sizes]**

7.4.2 Structural baseline results

To fill after execution.

- Total missing required sections across corpus: **[X]**
- Most frequently missing sections: **[Top 5]**
- Most frequent extra sections: **[Top 5]**

7.4.3 Cosine resolver contribution

To fill after execution.

- Total rescued mappings: **[X]**
- Reduction in missing-section count: **[before → after]**
- Representative rescued mappings: **[examples]**

7.4.4 Pass 1 results (integrity + semantic adequacy)

To fill after execution.

- Total integrity issues detected: **[X]**
- Total semantic issues detected: **[X]**
- Section score distribution summary: **[histogram summary]**

7.4.5 Pass 2 results (cross-view consistency)

To fill after execution.

- Entities extracted totals: **[counts per type]**
- Consistency issues total: **[X]**
- Breakdown by rule category: **[counts]**

7.5 Comparison of stages

This section synthesizes stage contribution:

- what baseline detects and where it over-reports due to naming variation,
- what cosine rescue reduces (false "missing"), and what it additionally detects (placeholders, template copies),
- what structure+similarity still misses that Pass 1 detects (semantic inadequacy),
- what Pass 1 still misses that Pass 2 detects (cross-view inconsistencies).

7.6 Qualitative analysis and expert feedback

A qualitative inspection complements quantitative metrics by assessing interpretability and industrial usefulness.

Proposed protocol (to execute if feasible).

- Sample **[k]** documents covering low/mid/high scores.
- For each document, inspect the top **[m]** findings by severity.
- Classify each finding as:
  - valid and useful,
  - valid but low priority,
  - false positive / ambiguous,
  - unclear evidence.

If feasible, practitioner feedback from a quality manager or architect is incorporated:

- perceived usefulness of evidence-backed findings,
- acceptance of the score rubric as a prioritization aid,
- suggestions for threshold tuning and rule adjustment.

7.7 Threats to evaluation validity

The evaluation is subject to several validity threats (discussed in depth in Chapter 8):

- **Construct validity:** proxy metrics (counts/scores) may not fully represent true documentation quality.
- **Internal validity:** threshold sensitivity and prompt formulation may influence outputs.
- **External validity:** results may depend on the company template variant and domain-specific writing conventions.
- **Reliability:** variability in audit passes is mitigated through bounded inputs, schema constraints, and evidence requirements.

7.8 Summary

This chapter evaluates the pipeline in terms of stage contribution, auditability, and industrial relevance. Results are reported quantitatively (issue counts, score distributions, and rule-category breakdowns) and qualitatively (evidence-backed examples and practitioner feedback). Chapter 8 interprets the results and discusses strengths, limitations, and industrial applicability.

8 Discussion

This chapter interprets the evaluation results from Chapter 7 and discusses the strengths, limitations, industrial applicability, and validity considerations of the proposed staged validation pipeline. The discussion is structured so that most arguments remain grounded in the design rationale and methodological constraints, while quantitative claims and illustrative examples are refined once the full results are available.

8.1 Summary of key findings

This section summarizes the main outcomes of the evaluation with a focus on stage contribution and practical usefulness.

Structural baseline findings. **[TO FILL AFTER RESULTS: key counts/patterns]**

Cosine rescue impact. **[TO FILL AFTER RESULTS: reduction in false "missing"]**

Pass 1 integrity and semantic adequacy. **[TO FILL AFTER RESULTS: dominant issue types, score distribution]**

Pass 2 cross-view consistency. **[TO FILL AFTER RESULTS: rule categories most frequently violated]**

This summary is intentionally concise (approximately one page) to provide a transition into the interpretation and implications discussed in the remainder of the chapter.

8.2 Strengths of the proposed approach

This section discusses strengths that follow directly from the pipeline design (Chapter 5) and implementation choices (Chapter 6), and that are supported by the evaluation evidence (Chapter 7).

S1 — Staged design improves acceptance and auditability. A central strength of the approach is the decomposition into deterministic checks, similarity-based alignment, and constrained semantic auditing. Deterministic stages offer reproducibility and high precision for structural completeness, while later stages add semantic depth without sacrificing auditability due to strict schema constraints and evidence requirements. This staged design enables incremental adoption: early stages can already provide value in reporting mode, while later stages can be introduced once organizational trust is established.

S2 — Robustness to naming variation without sacrificing explainability. The similarity layer improves robustness to practical documentation variation such as renamed headings or slightly modified section titles. Importantly, similarity is used as a mapping mechanism rather than as a proxy for quality. This preserves explainability: rescued mappings remain transparent through explicit correspondence pairs and similarity scores, and their impact on structural findings can be measured directly.

S3 — Guidance-aware semantic validation. By attaching template guidance to each section and heading, the audit shifts from generic text evaluation toward intent-based evaluation. Semantic findings can therefore be formulated as a mismatch between normative expectation (guidance) and the provided content. This makes findings interpretable as documentation defects rather than stylistic critique.

S4 — Evidence-backed outputs support industrial review workflows. The strict requirement that findings include verifiable evidence snippets increases trust and reduces the risk that the tool is perceived as a black-box classifier. Evidence-backed reporting supports practical review usage: reviewers can verify issues quickly and decide whether they require remediation, which is essential for adoption in time-constrained industrial workflows.

S5 — Cross-view consistency via intra-document traceability. Pass 2 supports consistency validation even when only text-centric documentation is available. By extracting architectural entities and checking their usage across complementary arc42 views (Building Block View, Runtime View, Deployment View, and rationale-related chapters), the approach approximates an intra-document traceability perspective without requiring formal architecture models. This addresses an industrially common scenario where architecture knowledge is primarily captured in narrative form.

Examples (to add after evaluation). [TO FILL AFTER RESULTS: Add 2–3 concrete examples demonstrating each strength, e.g., a rescued heading, a guidance mismatch with evidence, and a cross-view inconsistency case.]

8.3 Limitations

This section discusses limitations and failure modes in an engineering-oriented manner. The goal is to clarify what the approach does *not* guarantee and where misclassifications may occur.

L1 — Template dependence and organizational variation. Validation results are normative with respect to the company's arc42-derived reference template. Different organizations or domains may adapt arc42 differently (section selection, naming conventions, additional mandatory content), which would require updating the expected structure and guidance used by the pipeline.

L2 — False positives from heuristic integrity checks. Deterministic emptiness detection and similarity-based heuristics can misclassify legitimate concise content as incomplete, or flag domain-specific terminology as placeholders. While conservative thresholds reduce this risk, some tuning is typically necessary and should be informed by practitioner feedback and corpus characteristics.

L3 — Residual non-determinism and sensitivity in LLM-based judgments. Even with strict schemas, temperature settings, and evidence-only reporting, semantic judgments may vary across borderline cases and may be sensitive to prompt phrasing, model versions, and context budget decisions (e.g., truncation). The approach mitigates but cannot fully eliminate these effects, which is why run metadata and prompt/schema versioning are essential for traceability.

L4 — Limited understanding of diagrams and external artifacts. The current pipeline primarily evaluates textual content. Diagrams, UML artifacts, and external traceability files (e.g., unit mapping files) are not fully validated unless explicitly integrated. Consequently, a document may be semantically adequate in text but still contain inconsistent or low-quality diagrams that are not detected by the current approach.

L5 — Consistency checks limited to what is explicitly expressed in text. Entity-based consistency checks depend on explicit naming and consistent terminology. If building blocks or interfaces are implied rather than named, or if synonyms are used without clarification, cross-view rules may miss inconsistencies or may flag issues that are semantically valid but lexically divergent.

8.4 Industrial applicability and integration considerations

This section discusses how the approach can be used in practice and what integration strategies reduce organizational adoption risk.

8.4.1 Adoption path

A pragmatic adoption strategy is an incremental rollout:

1. **Reporting mode (non-blocking):** execute the pipeline and publish findings and aggregated statistics without enforcing blocking conditions.
2. **Assisted review:** reviewers focus on flagged sections using evidence snippets, reducing manual effort while maintaining human oversight.
3. **Quality gate mode (blocking):** enforce only high-confidence rules (e.g., missing mandatory sections or unresolved placeholders) once precision is validated and organizational acceptance is established.

8.4.2 Integration points

Two primary integration points are feasible:

- **Post-processing in an existing batch workflow:** execute the pipeline alongside existing crawlers or reporting tools to produce organization-wide documentation quality reports.
- **Earlier validation as a quality gate:** run deterministic checks (and optionally selected audit rules) in CI or code review to prevent low-quality documentation from entering repositories.

8.4.3 Practical concerns

Practical deployment requires addressing:

- **Runtime and cost:** deterministic stages are typically inexpensive, while LLM passes may introduce additional latency and cost, motivating staged configuration and selective execution.
- **Configuration management:** thresholds, template versions, prompt versions, and schema versions must be governed to preserve comparability over time.
- **Governance and ownership:** template evolution and validation rules require clear ownership (e.g., architecture governance team) to avoid misalignment between "expected" content and evolving practice.

8.5 Threats to validity

This section summarizes threats to validity following standard empirical software engineering reporting conventions.

8.5.1 Construct validity

The reported counts and scores are proxies for documentation quality and may not capture all relevant aspects (e.g., clarity, architectural correctness, or suitability of the architecture itself). Semantic adequacy judgments are relative to template guidance; if guidance is incomplete or overly generic, adequacy checks may inherit those limitations.

8.5.2 Internal validity

Findings can be influenced by threshold choices (e.g., cosine thresholds, emptiness criteria, duplication thresholds), by the prompt wording and schema design used in LLM passes, and by confounding factors such as document length and domain-specific writing style. Mitigations include versioned prompts and schemas, run metadata, and persisted intermediate artifacts enabling stage-by-stage analysis. Optional sensitivity checks can further quantify the impact of threshold changes. **[TO FILL AFTER RESULTS if performed]**

8.5.3 External validity

Generalization may be strongest for arc42-derived documentation in similar industrial contexts. Other documentation frameworks, different arc42 variants, or domains with substantially different terminology and documentation practices may require adaptation of the reference template, guidance extraction, and rule definitions.

8.5.4 Reliability

Deterministic stages are fully reproducible. For LLM-based stages, reliability is supported through temperature-controlled execution, bounded and guidance-aware inputs, strict schemas, and evidence-only reporting. However, model updates and platform changes can affect outputs; therefore, recording model identifiers and prompt/schema hashes in run metadata is necessary to support reproducible comparisons over time.

8.6 Ethical, privacy, and organizational considerations

Architecture documentation may contain sensitive technical details. Processing should respect confidentiality requirements and internal policies, and audit outputs should be stored and shared according to governance rules. Evidence snippets should be limited to what is necessary for verification, minimizing unnecessary exposure of proprietary content. If audit outputs are aggregated across units, access control becomes important to ensure that results are visible only to appropriate stakeholders.

8.7 Chapter summary

This chapter discussed strengths, limitations, and industrial applicability of the staged validation pipeline and analyzed threats to validity and confidentiality-related considerations. The next chapter concludes the thesis by summarizing contributions, answering the research questions, and outlining future extensions such as artifact-aware validation (e.g., diagrams/UML) and integration with additional traceability artifacts.

9 Conclusion and Future Work

9.1 Summary of the problem and approach

This thesis addressed the challenge of validating software architecture documentation quality in industrial settings where documentation is primarily text-centric, evolves over time, and is costly to review manually at scale. Although arc42(-derived) templates provide a normative structure, real-world documents frequently exhibit structural variation (e.g., renamed or reorganized headings), incomplete placeholder content (e.g., unfilled template slots or `TODO/TBD` markers), and inconsistencies across views. These factors motivate automated validation that goes beyond the mere presence of headings, while remaining explainable and auditable for industrial acceptance.

To address this, the thesis proposed and implemented a staged validation pipeline that combines deterministic structural validation, similarity-based alignment to improve robustness under naming variation, and constrained LLM-based auditing for integrity, semantic adequacy, and cross-view consistency. The approach operationalized documentation quality along four dimensions defined in Section 2.3: *structural completeness*, *integrity*, *semantic adequacy to section intent*, and *cross-view consistency*. Each stage of the pipeline is designed to contribute a specific capability while controlling a specific failure mode, and all audit findings are required to be evidence-backed.

9.2 Summary of contributions

This work makes the following contributions:

1. **An operational documentation quality model** for arc42(-derived) documentation, defining measurable dimensions for structural completeness, integrity, semantic adequacy, and cross-view consistency.
2. **A deterministic baseline checker** that validates structure and mandatory content presence against a reference template, producing explainable reports suitable for audit and aggregation.
3. **A similarity-based resolver** (TF-IDF with cosine similarity) that reduces false "missing section" findings caused by heading variation and identifies high-confidence integrity signals such as near-verbatim template reuse and unfilled placeholders.
4. **A guidance-aware preprocessing layer** that aligns document and template structure and attaches guidance to each audited section/heading, producing a bounded, audit-ready representation for semantic analysis.
5. **A two-pass auditing design**:
   - **Pass 1** for integrity and semantic adequacy, producing evidence-backed findings and section-level scores.
   - **Pass 2** for cross-view consistency via entity extraction and intra-document traceability rules across building blocks, runtime scenarios, deployment mappings, goals, decisions, and risks.
6. **Reproducibility- and evaluation-ready engineering packaging**, including versioned prompts and schemas, externalized configuration, persisted intermediate artifacts per stage, and run manifests with hashes for prompt/schema/config/template traceability.

9.3 Answer to the research questions

This section explicitly answers the research questions (RQ1–RQ4) defined in Chapter 1. Quantitative values and concrete examples are finalized once the evaluation results (Chapter 7) are available.

RQ1 (Structural): How effectively can a deterministic baseline checker validate arc42-derived structure and detect missing/extra headings in real industrial documents? The baseline checker demonstrated that deterministic template validation can reliably identify missing and extra structural elements and report them with high explainability and reproducibility. **[TO FILL AFTER RESULTS: key structural findings, e.g., most frequently missing sections/headings and corpus-level counts.]**

RQ2 (Robustness): How can similarity-based alignment reduce false "missing section" findings caused by heading renames and variations? The similarity-based resolver reduced false "missing section" detections introduced by naming variation by proposing transparent, inspectable mappings between template and document headings. This improved alignment of real documents to the normative template structure without using similarity as a quality proxy. **[TO FILL AFTER RESULTS: before/after reduction and representative rescued mapping examples.]**

RQ3 (Semantic & Integrity): To what extent can an LLM-based audit, constrained to evidence-backed outputs, detect stubs and assess semantic adequacy relative to arc42 guidance? The constrained Pass 1 audit enabled detection of incomplete content (stubs, placeholders, template remnants) and semantic mismatches where section content does not fulfill the intent expressed by template guidance. Auditability was maintained through schema-constrained outputs and evidence-backed reporting that grounds findings directly in the provided text. **[TO FILL AFTER RESULTS: dominant finding types, evidence coverage rate, score distribution patterns.]**

RQ4 (Consistency): How can cross-view consistency checks be operationalized on text-centric arc42 documentation without assuming formal architecture models? Pass 2 showed that cross-view consistency can be operationalized on text-only documentation by extracting architectural entities and validating their coherent usage across arc42 views (e.g., Building Block View vs. Runtime View vs. Deployment View), as well as alignment between goals, decisions, and risks when available. This realizes an intra-document traceability perspective without requiring external formal models. **[TO FILL AFTER RESULTS: most common inconsistency categories and representative issue types.]**

9.4 Practical implications

From an industrial perspective, the staged pipeline provides a realistic adoption path aligned with organizational constraints:

- **Low-risk reporting value immediately:** deterministic stages (baseline checks and preprocessing) can be deployed as reporting tools with minimal operational risk, producing structured outputs and aggregated statistics.
- **Gradual introduction of semantic auditing:** LLM-based auditing can be introduced incrementally once trust in evidence-backed outputs is established, beginning in non-blocking mode and later supporting stricter enforcement for high-confidence rules.
- **Scalable review support:** the approach reduces manual effort by directing reviewers to evidence-backed defects and by enabling aggregation across many documents, which is particularly valuable for batch crawler workflows and quality monitoring.

9.5 Limitations

Key limitations include dependence on the specific reference template variant and its guidance texts, limited coverage of non-textual artifacts (e.g., diagrams/UML) without explicit integration, and residual variability in semantic judgments despite constraints such as schema enforcement and evidence-only reporting. These limitations motivate the future work directions below.

9.6 Future work

This thesis opens several clear extensions:

FW1 — Artifact-aware validation (diagrams/UML). Extend validation beyond text to incorporate architecture diagrams and UML artifacts, for example by verifying the presence and completeness of required diagrams, extracting key labels/relationships when feasible, and checking consistency between textual descriptions and diagram content. Such extensions can improve coverage for architecture information that is commonly communicated visually.

FW2 — Integration with traceability artifacts (e.g., unit mapping). Integrate external mapping files (e.g., `unitmapping.json`) to strengthen traceability checks such as verifying that all units documented in the Building Block View exist in configuration management artifacts. This would connect documentation validation to governance artifacts and enable stronger industrial compliance rules.

FW3 — Trend analysis across versions. Extend the pipeline to compare audit outputs across document revisions to detect improvement/regression trends and support continuous quality monitoring. This would enable time-series reporting of documentation quality per unit and per arc42 section.

FW4 — Dashboard and workflow integration. Provide a lightweight dashboard that aggregates findings across documents and supports drill-down from statistics to evidence snippets, improving usability for quality managers and architects. Integration into existing review workflows can further increase adoption.

FW5 — Calibration and expert-in-the-loop refinement. Systematically calibrate thresholds, rules, and prompts based on structured reviewer feedback to reduce false positives and improve prioritization. This could include workflows for approving new rules, maintaining rule sets per domain, and establishing acceptance criteria for blocking checks.

FW6 — Generalization beyond arc42. Generalize the staged validation design to other architecture documentation frameworks by swapping the reference structure and guidance mappings while retaining the same pipeline philosophy (deterministic structure, robust alignment, constrained semantic auditing, evidence-backed reporting).

9.7 Closing statement

This thesis demonstrated that architecture documentation validation can be engineered as a staged pipeline that combines deterministic checks, similarity-based alignment, and constrained semantic auditing. By grounding validation in a normative reference template and enforcing evidence-backed, schema-constrained reporting, the approach supports scalable and auditable quality assurance for architecture documentation in industrial contexts. **[TO FILL AFTER RESULTS: final one-sentence result claim with concrete outcome wording.]**

Literature references